The Intuition Behind DP

Dynamic Programming is fundamentally about **remembering the past so you don't have to repeat it**.

Imagine I ask you: *What is 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1?*

You count them and say: *8*.

Then, I immediately add another "+ 1" to the end and ask: *What about now?*

You don't recount all the ones from the beginning. You just take your previous answer (8), add 1, and say: *9*.

That is DP in a nutshell. It is an optimization technique over plain recursion. Instead of solving the exact same subproblem multiple times, you solve it once, store the result, and reuse it whenever that subproblem appears again.

How to Identify a DP Problem

You cannot use DP on just any problem. A problem must exhibit two specific mathematical properties to be a candidate for DP:

- **Overlapping Subproblems:** The problem can be broken down into smaller, identical subproblems that are solved repeatedly. If a recursive algorithm recalculates the exact same branches over and over, you have overlapping subproblems.
- **Optimal Substructure:** The optimal solution to the main problem can be constructed directly from the optimal solutions of its subproblems. (e.g., the shortest path from A to C via B is made of the shortest path from A to B, and the shortest path from B to C).

If you read a CP problem and it asks for the **"maximum," "minimum," "longest," "shortest," or "number of ways"** to do something, and making a greedy choice at each step isn't guaranteed to be optimal, it is highly likely a DP problem.

The Classic Example: Fibonacci Sequence

Let's calculate the N-th Fibonacci number, where F(n) = F(n-1) + F(n-2), with base cases F(0) = 0nand F(1) = 1.

The Naive Recursive Approach

If you code this purely recursively in C++, it looks like this:

```
C++

int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

This is terrible for CP. The time complexity is O(2^n). To calculate fib(5), it calculates fib(3) twice, fib(2) three times, etc.

DP Approach 1: Top-Down (Memoization)

We keep the recursive structure but add a "cache" (an array or vector) to store answers. Before computing a state, we check if it is already in the cache.

```cpp
// Initialize cache with -1 to indicate "unsolved" states
vector<int> memo;

int fib_memo(int n) {
    if (n <= 1) return n;

    // If already calculated, return the cached value!
    if (memo[n] != -1) return memo[n];

    // Otherwise, calculate and store it before returning
    memo[n] = fib_memo(n - 1) + fib_memo(n - 2);
    return memo[n];
}

int main() {
    int n = 10;
    memo.assign(n + 1, -1); // Resize and fill with -1
    cout << "Fibonacci of " << n << " is: " << fib_memo(n) << "\n";
    return 0;
}
```

Time Complexity: O(N)

Space Complexity: O(N) for the memo array + O(N) for the recursion stack.

DP Approach 2: Bottom-Up (Tabulation)

Instead of starting from the top and recursing down, we start from our base cases and iteratively build our way up to the answer using a loop. This is usually preferred in CP because it avoids recursion overhead (no stack overflow risks) and is slightly faster.

```cpp
int fib_tab(int n) {
    if (n <= 1) return n;

    vector<int> dp(n + 1);

    // 1. Base cases
    dp[0] = 0;
    dp[1] = 1;

    // 2. Iteratively transition to build the answers
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}

int main() {
    int n = 10;
    cout << "Fibonacci of " << n << " is: " << fib_tab(n) << "\n";
    return 0;
}
```

Time Complexity: O(N)

Space Complexity: O(N) (can be optimized to O(1) by just keeping track of the last two variables, but an array is standard for general DP).

The 3-Step Framework for Solving Any DP Problem

When you encounter a new DP problem in a contest, do not rush to write code. Grab a pen and paper and define these three things:

1. **State Definition:** What does dp[i] (or dp[i][j]) actually represent in plain English? For example, " dp[i] is the minimum cost to reach step i ". If you get the state wrong, the rest of the problem will fail.

2. **State Transition:** How does dp[i] relate to smaller, previously calculated states? Find the mathematical formula. For example, dp[i] = min(dp[i-1], dp[i-2]) + cost[i].
3. **Base Cases:** What are the simplest, smallest subproblems? Usually, this is dp[0] or dp[1].

Now that you have the core intuition and the basic framework, the next step is applying it to different patterns (like 1D DP, 2D Grids, Knapsack, etc.).

Would you like to try solving a classic beginner CP problem together, like "Climbing Stairs" or the "Coin Change" problem, to test out this framework?